



Descent Phase Documentation

Scope: Complete documentation of the descent phase optimization system, including 3D dynamic programming with penalty guidance, minimum fuel descent trajectories, and integration with mission analysis for optimal fuel consumption during descent from cruise altitude to approach.

Table of Contents

1. [System Overview and Objectives](#)
2. [Mathematical Foundation](#)
3. [System Architecture and Data Structures](#)
4. [Minimum Mach Calculation](#)
5. [Dynamic Programming Optimization](#)
6. [Penalty System and Guidance](#)
7. [Code Execution Flow and Logic](#)
8. [Integration and Interface](#)
9. [Key Differences from Climb](#)

1) System Overview and Objectives

1.1 Purpose and Scope

The descent phase simulation implements a fuel-optimal descent trajectory optimization system that models aircraft performance during the descent segment from cruise altitude to approach altitude. The system employs 3D Dynamic Programming with integrated penalty guidance to generate physically realizable descent paths that minimize fuel consumption.

1.2 System Objectives

Primary Objectives:

- **Fuel-Optimal Descent:** Find minimum fuel consumption descent trajectories
- **Realistic Trajectories:** Generate physically achievable descent paths with appropriate Mach deceleration
- **Terminal Constraints:** Achieve target approach speed (Mach 0.25) at approach altitude (300m)
- **Mission Integration:** Provide seamless connection with climb and cruise phases

Key Components:

- 3D dynamic programming optimization (Mach × Altitude × Lever)
- Penalty-based guidance system for Mach trajectory realism
- Dynamic minimum Mach calculation based on stall speed and weight
- Engine performance integration with idle-focused thrust range
- Atmospheric property calculations using ISA model
- Integration with aerodynamics tables from Excel

1.3 System Flow Overview

The descent simulation follows a logical progression:

1. **Initialization:** Extract initial state from cruise phase results
 2. **Grid Setup:** Create discrete state space for DP optimization
 3. **DP Optimization:** Execute backward dynamic programming with penalty guidance
 4. **Trajectory Extraction:** Backtrack optimal path from cost matrix
 5. **Performance Calculation:** Compute detailed trajectory data (time, fuel, forces)
 6. **Results Analysis:** Generate comprehensive performance data and visualizations
-

2) Mathematical Foundation

2.1 Descent Flight Physics

Theory: Descent flight requires negative specific excess power, where drag exceeds thrust, causing the aircraft to dissipate potential and kinetic energy. The descent rate and fuel consumption are determined by the thrust-drag imbalance and airspeed.

Mathematical Formulation:

$$P_s = \frac{(T_{total} - D) \times V_{TAS}}{W}$$

$$\frac{dh}{dt} = P_s \quad (\text{for quasi-steady descent, } P_s < 0)$$

$$J = \frac{\dot{m}_{fuel}}{|P_s|} \quad [kg/m]$$

Where:

- P_s = Specific excess power (m/s), negative for descent
- $T_{\{total\}}$ = Total thrust (N)
- D = Total drag (N)
- $V_{\{TAS\}}$ = True airspeed (m/s)
- W = Aircraft weight (N)
- J = Fuel cost density (kg/m), fuel consumed per meter of altitude lost

Critical Difference from Climb:

- **Descent:** $P_s < 0$ (energy dissipation), $T < D$ typically
- **Climb:** $P_s > 0$ (energy addition), $T > D$ required

2.2 Dynamic Programming Formulation

Theory: The descent optimization problem is formulated as a discrete-time dynamic programming problem using Bellman's principle of optimality, with the objective of minimizing total fuel consumption while descending from cruise altitude to approach altitude.

Continuous Problem Formulation:

$$\min_{\ell(\cdot), M(\cdot)} J_{tot} = \int_{h_0}^{h_f} \underbrace{\frac{\dot{m}(\ell, M, h)}{|P_s(\ell, M, h)|}}_{J(\ell, M, h) \text{ [kg/m]}} dh$$

Where:

- h_0 = Initial cruise altitude (e.g., 10,000m)

- h_f = Final approach altitude (300m)
- $\ell(h)$ = Throttle lever position as function of altitude
- $M(h)$ = Mach number as function of altitude

Discrete State Space:

$$S = \{M_i\} \times \{h_j\} \times \{\ell_k\}$$

Bellman Equation (Backward Formulation):

$$F[k+1, i', j'] = \min_{i,j} \left\{ F[k, i, j] + \frac{1}{2} (J[k, i, j] + J[k+1, i', j']) \times |\Delta h| \right\}$$

Where:

- $F[k, i, j]$ = Minimum fuel cost to reach altitude level k with Mach i and lever j
- Δh = Altitude step (negative for descent)
- Trapezoidal integration used for step cost calculation

2.3 Stall Speed and Minimum Mach

Theory: Aircraft cannot fly below stall speed without loss of lift. Minimum safe Mach must account for stall speed with safety margin.

Stall Speed Formula:

$$V_{stall} = \sqrt{\frac{2W}{\rho S_{ref} C_{L,max}}}$$

Minimum Safe Mach:

$$M_{min} = \frac{V_{stall} \times \text{safety_margin}}{a(h)}$$

Where:

- W = Aircraft weight (N)
- $\rho(h)$ = Air density at altitude (kg/m³)
- S_{ref} = Reference wing area (m²)
- $C_{L,max}$ = Maximum lift coefficient

- `a(h)` = Speed of sound at altitude (m/s)
- `safety_margin` = 1.3 (30% above stall)

Altitude Dependence:

As the aircraft descends into denser air, the minimum Mach decreases, allowing lower airspeeds at lower altitudes.

3) System Architecture and Data Structures

3.1 Core System Parameters

Descent Configuration:

```
class DescentConfiguration:
    # Descent targets
    TARGET_DESCENT_ALT_M = 300.0          # Approach altitude (~1000 ft)
    TARGET_APPROACH_MACH = 0.25           # Target Mach at approach

    # Grid resolution for DP optimization
    DESCENT_MACH_SAMPLES = 41             # Number of Mach samples for DP grid
    DESCENT_ALT_SAMPLES = 50              # Number of altitude samples for DP grid
    DESCENT_LEVER_SAMPLES = 11            # Number of lever samples for DP grid

    # Speed constraints
    STALL_SPEED_SAFETY_MARGIN = 1.3        # Safety margin above stall (1.3 = 30%)
    ABSOLUTE_MIN_DESCENT_MACH = 0.15      # Absolute minimum Mach (safety fallback)
    MAX_DESCENT_MACH = M_MMO               # Maximum Mach for descent (0.94)
```

Shared Aircraft Parameters:

```
N_ENGINES = 2                          # Number of engines
INITIAL_MASS_KG = 65000.0               # Take-off mass (kg)
S_REF_M2 = 122.4                        # Reference wing area (m²)
M_MMO = 0.94                            # Maximum operating Mach
CL_MAX = None                           # Maximum lift coefficient (from Excel)
```

3.2 Data Structures

3.2.1 DescentInitialState

Purpose: Container for initial conditions extracted from cruise phase results.

Structure:

```
@dataclass
class DescentInitialState:
    altitude_m: float          # Starting cruise altitude [m]
    mach: float                # Starting cruise Mach number
    weight_kg: float           # Aircraft weight after cruise [kg]
    fuel_consumed_total_kg: float  # Cumulative fuel (climb + cruise) [kg]
    total_time_s: float        # Cumulative time (climb + cruise) [s]

    def __post_init__(self):
        """Validate initial descent state."""
        if self.altitude_m <= TARGET_DESCENT_ALT_M:
            raise ValueError("Descent altitude must be above target")
        if not (0.2 <= self.mach <= M_MMO):
            raise ValueError("Descent Mach outside safe range")
        if self.weight_kg <= 0:
            raise ValueError("Aircraft weight must be positive")
```

3.2.2 DescentResults

Purpose: Complete results container for descent DP optimization (analogous to MinFuelSchedule from climb).

Structure:

```

@dataclass
class DescentResults:
    # Metadata
    strategy_name: str = "3D DP Optimal Descent (with Penalty Guidance)"

    # Trajectory arrays
    alt_m: np.ndarray = # Altitude profile [m]
    mach: np.ndarray = # Mach profile
    lever: np.ndarray = # Lever profile [0-1]
    cumFuel_kg: np.ndarray = # Cumulative fuel consumed [kg]
    dt_s: np.ndarray = # Time steps [s]
    dFuel_kg: np.ndarray = # Fuel increments [kg]

    # Performance arrays
    thrust_total_N: np.ndarray = # Total thrust [N]
    drag_N: np.ndarray = # Drag force [N]
    fuel_flow_kgps: np.ndarray = # Fuel flow rate [kg/s]
    descent_rate_mps: np.ndarray = # Descent rate [m/s]
    temperature_K: np.ndarray = # Temperature [K]
    density_kgpm3: np.ndarray = # Air density [kg/m³]
    true_airspeed_mps: np.ndarray = # True airspeed [m/s]
    specific_excess_power_mps: np.ndarray = # Specific excess power [m/s]

    # Time and weight evolution
    time_s: np.ndarray = # Time array [s]
    weight_kg: np.ndarray = # Weight evolution [kg]

    # Summary statistics
    total_time_s: float = # Total descent time [s]
    total_fuel_consumed_kg: float = # Total fuel consumed [kg]
    final_weight_kg: float = # Final aircraft weight [kg]
    average_descent_rate_mps: float = # Average descent rate [m/s]
    average_fuel_flow_kgps: float = # Average fuel flow [kg/s]

    # Initial and target states
    initial_altitude_m: float = # Initial cruise altitude [m]
    initial_mach: float = # Initial cruise Mach
    initial_weight_kg: float = # Initial descent weight [kg]
    target_altitude_m: float = # Target approach altitude [m]

```

```

target_mach: float                                # Target approach Mach

def get_summary_dict(self) -> Dict[str, Any]:
    """Get summary statistics as dictionary."""
    return {
        'strategy': self.strategy_name,
        'descent_altitude_change_m': self.initial_altitude_m - self.target_altitude_m,
        'descent_time_minutes': self.total_time_s / 60.0,
        'descent_fuel_kg': self.total_fuel_consumed_kg,
        'avg_descent_rate_fpm': self.average_descent_rate_mps * 196.85,
        'initial_altitude_m': self.initial_altitude_m,
        'final_altitude_m': self.target_altitude_m,
        'initial_mach': self.initial_mach,
        'final_mach': self.target_mach,
    }

```

3.3 System Integration Components

Aerodynamics System:

```

class AeroTables:
    def get_drag(self, mach: float, altitude_m: float) -> float
    def get_cl(self, mach: float, altitude_m: float) -> float

```

Engine System:

```

class EngineWrapper:
    def thrust_with_lever(self, lever: float, mach: float, altitude_m: float) -> float
    def tsfc_current(self) -> float

```

Atmosphere System:

```

def isa_properties(h_m: float) -> Tuple[float, float, float]
def a_from_altitude(h_m: float) -> float

```

4) Minimum Mach Calculation

4.1 Stall-Based Minimum Mach

Function: `calculate_min_descent_mach()`

Purpose: Calculate minimum safe Mach number for descent based on aerodynamic stall speed with safety margin. This ensures the aircraft never flies below safe operating speeds during descent.

Mathematical Implementation:

```

def calculate_min_descent_mach(altitude_m: float, weight_kg: float,
                               cl_max: Optional[float] = None,
                               s_ref_m2: Optional[float] = None,
                               safety_margin: float = None) -> float:
    """
    Calculate minimum safe Mach number for descent based on stall speed.

    Uses aerodynamic stall speed formula with safety margin:
    V_stall = sqrt(2 * W / (rho * S_ref * CL_max))
    V_min = V_stall * safety_margin
    M_min = V_min / speed_of_sound

    Args:
        altitude_m: Current altitude in meters
        weight_kg: Current aircraft weight in kg
        cl_max: Maximum lift coefficient (defaults to CL_MAX from climb module)
        s_ref_m2: Reference wing area in m² (defaults to S_REF_M2)
        safety_margin: Safety factor above stall speed (defaults to 1.3)

    Returns:
        float: Minimum safe Mach number for the given conditions
    """

    # Get atmospheric properties
    T, p, rho = isa_properties(altitude_m)
    a = a_from_altitude(altitude_m)

    # Calculate stall speed: V_stall = sqrt(2*W/(rho*S*CL_max))
    weight_N = weight_kg * G_C
    q_min = weight_N / (s_ref_m2 * cl_max) # Minimum dynamic pressure
    v_stall_mps = np.sqrt(2 * q_min / rho) # Stall speed in m/s

    # Apply safety margin
    v_min_mps = v_stall_mps * safety_margin

    # Convert to Mach number
    m_min = v_min_mps / a

    # Apply reasonable bounds [0.15, 0.40]

```

```
m_min_bounded = np.clip(m_min,
                        DescentConfiguration.ABSOLUTE_MIN_DESCENT_MACH,
                        0.40)

return float(m_min_bounded)
```

4.2 Key Assumptions

Physical Assumptions:

1. **Constant CL_{max}**: Maximum lift coefficient assumed independent of Mach and altitude
2. **Safety margin**: 30% buffer above stall speed accounts for gusts, maneuvers, uncertainties
3. **Constant weight**: Uses initial descent weight for all calculations (fuel burn during descent is small)
4. **Quasi-steady flight**: Stall speed formula assumes steady, level flight conditions

Bounded Results:

- **Lower bound**: 0.15 Mach (safety fallback for extreme conditions)
- **Upper bound**: 0.40 Mach (prevents unrealistic high minimum speeds)

4.3 Altitude Dependence

Physical Behavior:

```
High altitude (10,000m):
- Low air density ( $\rho \approx 0.41 \text{ kg/m}^3$ )
- Higher true airspeed required for same lift
-  $M_{\min} \approx 0.35\text{-}0.40$ 

Low altitude (1,000m):
- High air density ( $\rho \approx 1.11 \text{ kg/m}^3$ )
- Lower true airspeed sufficient for lift
-  $M_{\min} \approx 0.20\text{-}0.25$ 
```

This decreasing trend with altitude is **opposite to climb**, where minimum Mach often increases with altitude due to CL_{max} variations and engine limits.

5) Dynamic Programming Optimization

5.1 3D State Space Definition

State Variables:

- **Altitude:** $h \in [h_{\text{cruise}}, h_{\text{approach}}]$ (descending from high to low)
- **Mach Number:** $M \in [M_{\text{min}}(h), M_{\text{MMO}}]$ (dynamic minimum based on altitude)
- **Engine Lever:** $\ell \in [0.0, 0.3]$ (idle to low thrust, focused on descent regime)

Discretization:

```
# Altitude grid (descending)
H_descent = np.linspace(initial_altitude_m, target_altitude_m, DESCENT_ALT_SAMPLES)
# Example: np.linspace(10000, 300, 50) → 50 altitude levels

# Mach grid
M_min_start = calculate_min_descent_mach(initial_altitude_m, initial_weight_kg)
M_max = min(0.85, initial_mach + 0.05) # Slightly above cruise Mach
M_grid = np.linspace(M_min_start, M_max, DESCENT_MACH_SAMPLES)
# Example: np.linspace(0.25, 0.85, 41) → 41 Mach levels

# Lever grid (focused on idle/low thrust for descent)
lever_grid = np.linspace(0.0, 0.3, DESCENT_LEVER_SAMPLES)
# Example: np.linspace(0.0, 0.3, 11) → 11 lever levels (idle to 30%)
```

Grid Size:

- Total states: $50 \times 41 \times 11 = 22,550$ states
- Computational complexity: $O(K \times I^2 \times L^2)$ where K =altitude levels, I =Mach levels, L =lever levels

5.2 Bellman's Principle Implementation

Function: `DescentCore.DynamicProgrammingOptimizer.solve_descent_dp()`

Algorithm Structure:

```

@staticmethod
def solve_descent_dp(aero: AeroTables, eng: EngineWrapper,
                    M_grid: np.ndarray, H_sched: np.ndarray,
                    initial_state: DescentInitialState,
                    lever_samples: int = 11,
                    target_mach: float = TARGET_APPROACH_MACH,
                    target_mach_tolerance: float = 0.015):
    """
    3D Dynamic Programming solver for minimum fuel descent optimization.

    Forward dynamic programming from cruise altitude (high) to approach altitude (low),
    with penalty guidance to ensure realistic Mach trajectories.

    Returns:
        DescentResults: Optimal descent schedule
        dict: Additional information (costs, path, etc.)
    """

    K, I = len(H_sched), len(M_grid) # Altitude levels, Mach levels
    L = lever_samples                 # Lever levels

    # Create lever grid (descent typically uses low thrust: 0-30%)
    lever_grid = np.linspace(0.0, 0.3, L)

    # Initialize 3D cost matrix and predecessor array
    F = np.full((K, I, L), np.inf) # Cost-to-go
    prv = np.full((K, I, L, 3), -1, dtype=int) # Predecessor [k, i, j]

    # Set starting point (from cruise altitude)
    start_mach_idx = np.argmin(np.abs(M_grid - initial_state.mach))
    start_lever_idx = 0 # Start at idle
    F[0, start_mach_idx, start_lever_idx] = 0.0 # Starting cost is 0

    # Forward pass - 3D Dynamic Programming (descending)
    for k in range(K - 1): # For each altitude level
        current_alt = H_sched[k]
        next_alt = H_sched[k + 1]
        dh = next_alt - current_alt # Negative for descent

```

```
# Calculate descent fraction for penalty system
descent_fraction = k / (K - 1.0) if K > 1 else 0.0

# Find all feasible current states
feasible_states = np.where(np.isfinite(F[k]))

for state_idx in range(len(feasible_states[0])):
    i = feasible_states[0][state_idx] # Mach index
    j = feasible_states[1][state_idx] # Lever index

    current_mach = M_grid[i]
    current_lever = lever_grid[j]

    # Consider neighboring states (5x5 grid in Mach-Lever space)
    for di in [-2, -1, 0, 1, 2]: # Mach change
        for dj in [-2, -1, 0, 1, 2]: # Lever change
            next_mach_idx = i + di
            next_lever_idx = j + dj

            # Check bounds
            if (0 <= next_mach_idx < I and
                0 <= next_lever_idx < L):

                next_mach = M_grid[next_mach_idx]
                next_lever = lever_grid[next_lever_idx]

                # Calculate dynamic minimum Mach at next altitude
                min_mach_next = calculate_min_descent_mach(
                    next_alt, initial_state.weight_kg
                )

                # Check feasibility
                if (next_mach >= min_mach_next and
                    next_mach <= M_MMO):

                    # Compute fuel costs WITH PENALTIES
                    current_cost = DescentCore.compute_descent_cost(
                        aero, eng, current_alt, current_mach,
                        current_lever, initial_state.weight_kg,
                        target_mach, descent_fraction
                    )
```

```

    )
    next_cost = DescentCore.compute_descent_cost(
        aero, eng, next_alt, next_mach,
        next_lever, initial_state.weight_kg,
        target_mach, (k+1)/(K-1.0)
    )

    if (np.isfinite(current_cost) and
        np.isfinite(next_cost) and
        current_cost > 0 and next_cost > 0):

        # Trapezoidal integration for fuel cost
        step_cost = 0.5 * (current_cost + next_cost) * abs(dh)
        total_cost = F[k, i, j] + step_cost

        # Update if this path is better
        if total_cost < F[k + 1, next_mach_idx, next_lever_idx]:
            F[k + 1, next_mach_idx, next_lever_idx] = total_cost
            prv[k + 1, next_mach_idx, next_lever_idx] = [k, i, j]

# Apply terminal Mach constraint
valid_final = np.abs(M_grid - target_mach) < target_mach_tolerance
for i in range(I):
    if not valid_final[i]:
        F[-1, i, :] = np.inf

# Find optimal final state
final_flat_idx = np.nanargmin(F[-1])
final_mach_idx, final_lever_idx = np.unravel_index(final_flat_idx, F[-1].shape)

# Backtrack to find optimal path
path = backtrack_optimal_path(F, prv, final_mach_idx, final_lever_idx, ...)

return descent_result, info

```

5.3 Cost Function Calculation

Function: `DescentCore.compute_descent_cost()`

Detailed Implementation:

```

@staticmethod
def compute_descent_cost(aero: AeroTables, eng: EngineWrapper,
                        altitude: float, mach: float, lever: float,
                        mass_kg: float,
                        target_mach: float = TARGET_APPROACH_MACH,
                        descent_fraction: float = None) -> float:
    """
    Compute fuel cost density for descent WITH PENALTIES.

    Cost function:  $J = \dot{m} / |P_s| + \text{Mach\_penalty} + \text{Lever\_penalty}$ 

    Args:
        aero: Aerodynamics tables
        eng: Engine wrapper
        altitude: Altitude in meters
        mach: Mach number
        lever: Throttle lever position (0-1)
        mass_kg: Aircraft mass in kg
        target_mach: Target Mach for penalty calculation
        descent_fraction: Descent progress (0.0 = start, 1.0 = target)

    Returns:
        float: Total cost density (fuel + penalties) in kg/m, or inf if infeasible
    """
    try:
        # Get atmospheric properties
        a = a_from_altitude(altitude)
        V = mach * a

        # Get thrust
        T_per = eng.thrust_with_lever(lever, mach, altitude)
        if T_per is None or not np.isfinite(T_per) or T_per < 0:
            return np.inf
        T_tot = T_per * N_ENGINES

        # Get drag
        D = aero.get_drag(mach, altitude)
        if not np.isfinite(D) or D < 0:
            return np.inf

```



```

# Calculate specific excess power
W = mass_kg * G_C
Ps = ((T_tot - D) * V) / W

# For descent, Ps should be negative (energy dissipation)
if Ps >= 0: # Can't descend with positive/zero Ps
    return np.inf

# Get fuel flow
tsfc = eng.tsfc_current()
if tsfc is None or not np.isfinite(tsfc) or tsfc < 0:
    return np.inf
mdot = tsfc * T_per * N_ENGINES

# Base fuel cost density (fuel per meter of descent)
J = mdot / abs(Ps)

if not np.isfinite(J) or J <= 0:
    return np.inf

# Add Mach penalty if guidance is enabled
if target_mach is not None and DescentCore.PenaltySystem.MACH_TRAJECTORY_GUIDANCE:
    mach_penalty = DescentCore.PenaltySystem.compute_mach_penalty(
        mach, target_mach, None, descent_fraction
    )
    J += mach_penalty

# Add lever penalty if guidance is enabled
if DescentCore.PenaltySystem.LEVER_PENALTY_GUIDANCE:
    lever_penalty = DescentCore.PenaltySystem.compute_lever_penalty(
        lever, descent_fraction
    )
    J += lever_penalty

return J

except Exception:
    return np.inf

```

Key Points:

1. **Negative P_s required:** Descent requires $P_s < 0$, otherwise state is infeasible
 2. **Absolute value:** Cost uses $|P_s|$ to ensure positive cost density
 3. **Integrated penalties:** Mach and lever penalties added directly to cost
 4. **Feasibility checks:** Multiple checks ensure physical validity of state
-

6) Penalty System and Guidance

6.1 Mach Trajectory Guidance

Purpose: Guide optimization toward realistic Mach deceleration profiles that ensure the target approach Mach (0.25) is achievable with reasonable deceleration rates.

Conceptual Framework: Creates a **reachability corridor** that dynamically narrows as descent progresses, ensuring the aircraft can reach the target Mach by the final altitude.

Penalty Function:

```

@staticmethod
def compute_mach_penalty(current_mach: float, target_mach: float,
                        prev_mach: float = None,
                        descent_fraction: float = None) -> float:
    """
    Compute Mach penalty using reachability-constrained approach FOR DESCENT.

    INVERTED FROM CLIMB: For descent, target is LOW Mach (0.25) at LOW altitude (300m).
    Creates a dynamic safety corridor that ensures target remains achievable.

    Args:
        current_mach: Current Mach number
        target_mach: Final target Mach number (0.25 for approach)
        prev_mach: Previous Mach number (unused - kept for API compatibility)
        descent_fraction: Fraction of descent progress (0.0 = start, 1.0 = target)

    Returns:
        penalty: Penalty value in kg per meter
    """
    if descent_fraction is None:
        descent_fraction = 0.0

    # Calculate remaining descent fraction and steps
    remaining_fraction = 1.0 - descent_fraction
    estimated_steps_remaining = (remaining_fraction *
                                DescentCore.PenaltySystem.TOTAL_DESCENT_STEPS_ESTIMATE)

    # Calculate maximum achievable Mach change with reasonable rates
    max_achievable_change = (DescentCore.PenaltySystem.MAX_REASONABLE_MACH_RATE *
                              estimated_steps_remaining)

    # Define reachability corridor bounds
    # For descent: target is LOWER than start, so corridor is around target
    min_reachable_mach = target_mach - max_achievable_change
    max_reachable_mach = target_mach + max_achievable_change

    # Calculate urgency factor (increases as we approach target altitude)
    urgency = ((1.0 - remaining_fraction) *
                DescentCore.PenaltySystem.URGENCY_MULTIPLIER)

```

```

# Apply penalties based on position relative to corridor
if current_mach < min_reachable_mach:
    # Below corridor - too slow, risk of stall
    deviation = min_reachable_mach - current_mach
    penalty = (urgency * DescentCore.PenaltySystem.MACH_PENALTY_BASE_WEIGHT *
               (deviation ** 2))

elif current_mach > max_reachable_mach:
    # Above corridor - too fast, won't slow down in time
    deviation = current_mach - max_reachable_mach
    penalty = (urgency * DescentCore.PenaltySystem.MACH_PENALTY_BASE_WEIGHT *
               (deviation ** 2))

else:
    # Within corridor - apply progressive guidance toward target
    if descent_fraction > 0.7:
        # Strong final phase guidance (70-100% descent)
        final_phase_strength = (descent_fraction - 0.7) / 0.3
        mach_deviation = current_mach - target_mach

        # Extra penalty boost for final 10% of descent
        if descent_fraction > 0.9:
            final_boost = ((descent_fraction - 0.9) / 0.1) * 2.0
            final_phase_strength *= (1.0 + final_boost)

        penalty = (final_phase_strength *
                   DescentCore.PenaltySystem.GUIDANCE_PENALTY_WEIGHT *
                   (mach_deviation ** 2))
    else:
        penalty = 0.0 # No penalty in early descent phase

return penalty

```

Key Constants:

```
MACH_PENALTY_BASE_WEIGHT = 0.3           # Base penalty weight
MAX_REASONABLE_MACH_RATE = 0.02          # Max Mach change per optimization step
TOTAL_DESCENT_STEPS_ESTIMATE = 50         # Expected DP grid steps
URGENCY_MULTIPLIER = 2.0                  # Urgency scaling with progress
GUIDANCE_PENALTY_WEIGHT = 0.5            # Strong guidance penalty in corridor
```

Penalty Regions:

- 1. **Below Corridor** (Mach too low):
 - Risk: Stall, insufficient energy
 - Penalty: Quadratic with deviation, scaled by urgency
- 2. **Above Corridor** (Mach too high):
 - Risk: Cannot decelerate in time, overshoot target
 - Penalty: Quadratic with deviation, scaled by urgency
- 3. **Within Corridor** (Acceptable range):
 - Early phase (0-70%): No penalty (flexibility)
 - Late phase (70-100%): Progressive guidance toward target
 - Final phase (90-100%): Strong enforcement with 2× boost

Critical Difference from Climb:

Aspect	Climb	Descent
Target location	High Mach at high altitude	Low Mach at low altitude
Corridor orientation	Opens upward (accelerating)	Narrows downward (decelerating)
Physical challenge	Acceleration while climbing	Deceleration while descending
Penalty logic	Encourage speed gain	Encourage speed loss

6.2 Lever Position Penalties

Purpose: Penalize excessive throttle usage (lever > 85%) to guide optimizer toward fuel-efficient idle descent while maintaining realistic engine operation limits.

Penalty Function:

```

@staticmethod
def compute_lever_penalty(current_lever: float,
                          descent_fraction: float = None) -> float:
    """
    Compute penalty for high lever positions (SAME AS CLIMB).

    For descent, low thrust (idle, 0-20%) is desired, so penalizing high lever
    (>85%) naturally guides optimizer toward fuel-efficient idle descent.

    Engine limits are altitude-independent - high thrust settings cause same
    thermal and mechanical stress regardless of altitude or flight phase.

    Real-world considerations:
    - 0-20% lever = Idle to low thrust (descent typical)
    - 85% lever = Maximum Continuous Thrust (MCT) - unlimited duration
    - 90%+ lever = Takeoff/Go-around thrust - limited duration, high wear
    - 95%+ lever = Maximum Takeoff Thrust - emergency use only

    Args:
        current_lever: Current lever position (0.0 to 1.0)
        descent_fraction: Unused parameter (kept for backward compatibility)

    Returns:
        penalty: Penalty value in kg (altitude-independent)
    """
    penalty = 0.0

    # Only apply penalty if lever exceeds MCT threshold (85%)
    if current_lever > DescentCore.PenaltySystem.LEVER_PENALTY_THRESHOLD:
        # Calculate excess lever above MCT threshold
        excess_lever = current_lever - DescentCore.PenaltySystem.LEVER_PENALTY_THRESHOLD

        # Base penalty using exponential curve
        lever_penalty = excess_lever ** DescentCore.PenaltySystem.LEVER_PENALTY_EXPONENT

        # Apply critical penalty for very high lever positions (90%+)
        if current_lever > DescentCore.PenaltySystem.LEVER_PENALTY_CRITICAL_THRESHOLD:
            critical_excess = (current_lever -
                               DescentCore.PenaltySystem.LEVER_PENALTY_CRITICAL_THRESHOLD)

```

```

critical_penalty = critical_excess ** (
    DescentCore.PenaltySystem.LEVER_PENALTY_EXPONENT + 1.0
)
lever_penalty += (critical_penalty *
    DescentCore.PenaltySystem.LEVER_PENALTY_CRITICAL_MULTIPLIER)

# Apply ultra-critical penalty for maximum thrust positions (95%+)
if current_lever > DescentCore.PenaltySystem.LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD:
    ultra_critical_excess = (
        current_lever -
        DescentCore.PenaltySystem.LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD
    )
    ultra_critical_penalty = ultra_critical_excess ** (
        DescentCore.PenaltySystem.LEVER_PENALTY_EXPONENT + 2.0
    )
    lever_penalty += (ultra_critical_penalty *
        DescentCore.PenaltySystem.LEVER_PENALTY_ULTRA_CRITICAL_MULTIPLIER)

# Use constant penalty weight - engine limits are altitude-independent
penalty_weight = DescentCore.PenaltySystem.LEVER_PENALTY_WEIGHT
penalty = penalty_weight * lever_penalty

return penalty

```

Key Constants:

LEVER_PENALTY_WEIGHT = 3.0	# Base weight for lever penalty
LEVER_PENALTY_THRESHOLD = 0.85	# MCT threshold (85%)
LEVER_PENALTY_EXPONENT = 3.0	# Cubic penalty curve
LEVER_PENALTY_CRITICAL_THRESHOLD = 0.90	# Critical threshold (90%)
LEVER_PENALTY_CRITICAL_MULTIPLIER = 5.0	# Critical range multiplier
LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD = 0.95	# Ultra-critical threshold (95%)
LEVER_PENALTY_ULTRA_CRITICAL_MULTIPLIER = 20.0	# Ultra-critical multiplier

Penalty Structure:

Lever Range	Description	Penalty
0-85%	Normal operation (idle to MCT)	0 (no penalty)
85-90%	Above MCT, entering high-wear region	Base cubic penalty
90-95%	Takeoff/go-around thrust region	Base + 5× critical penalty
95-100%	Maximum takeoff thrust	Base + 5× critical + 20× ultra-critical

Why Same as Climb:

The lever penalty is **identical** in both climb and descent because:

1. **Engine limits are physical:** Thermal and mechanical stresses depend only on lever position, not flight phase
2. **Certification limits:** FAA/EASA limits on high thrust duration apply equally to all phases
3. **Maintenance considerations:** High thrust usage increases wear regardless of whether climbing or descending

Effect in Descent:

Since descent typically requires very low thrust (0-20%), the optimizer rarely encounters the penalty region. However, the penalty provides a "soft constraint" that prevents unrealistic high-thrust descent profiles.

6.3 Integrated Cost Function

Total Cost Calculation:

```
J_total = J_base + P_mach + P_lever

Where:
J_base = mdot / |Ps|           [kg/m] - Base fuel cost
P_mach = f(M, M_target, ...)  [kg/m] - Mach guidance penalty
P_lever = g(lever)            [kg/m] - Lever position penalty
```

Unified Units: All components have units of [kg/m], allowing direct addition.

7) Code Execution Flow and Logic

7.1 System Entry Point and Initialization

Main Entry Function: `run_descent_dp_optimization()` in `descent.py`

Actual Execution Sequence:

```

def run_descent_dp_optimization(cruise_results: CruiseResults,
                                climb_fuel_kg: float,
                                climb_time_s: float,
                                aero: AeroTables,
                                engine: EngineWrapper,
                                target_altitude_m: float = TARGET_DESCENT_ALT_M,
                                target_mach: float = TARGET_APPROACH_MACH,
                                n_altitude_steps: int = 50,
                                n_mach_samples: int = 41,
                                lever_samples: int = 11) -> Tuple[DescentResults, Dict]:
    """
    Run 3D Dynamic Programming optimization for descent with penalty guidance.

    Main interface function similar to climb's solve_3d_fixed_mass.
    """

    print(f"\n{'='*80}")
    print("3D DYNAMIC PROGRAMMING DESCENT OPTIMIZATION (with Penalty Guidance)")
    print(f"{'='*80}")
    print(f"Target: Mach {target_mach:.3f} at {target_altitude_m:.0f}m altitude")
    print(f"Penalty System: Mach guidance + Lever penalties (same as climb)")
    print(f"{'='*80}")

    # STEP 1: Extract initial state
    initial_state = extract_descent_initial_state(
        cruise_results, climb_fuel_kg, climb_time_s
    )

    # STEP 2: Create descent altitude schedule (from high to low)
    H_descent = np.linspace(initial_state.altitude_m,
                             target_altitude_m,
                             n_altitude_steps)

    # STEP 3: Calculate dynamic minimum Mach at highest altitude
    min_mach_start = calculate_min_descent_mach(
        initial_state.altitude_m,
        initial_state.weight_kg
    )

```

```

# STEP 4: Create Mach grid
M_max = min(0.85, initial_state.mach + 0.05)
M_min = max(min_mach_start, target_mach - 0.1)
M_grid = np.linspace(M_min, M_max, n_mach_samples)

# STEP 5: Run DP optimization
dp_result, dp_info = DescentCore.DynamicProgrammingOptimizer.solve_descent_dp(
    aero=aero,
    eng=engine,
    M_grid=M_grid,
    H_sched=H_descent,
    initial_state=initial_state,
    lever_samples=lever_samples,
    target_mach=target_mach,
    target_mach_tolerance=DescentCore.PenaltySystem.TARGET_MACH_TOLERANCE
)

print(f"{'='*80}")
print("DP OPTIMIZATION COMPLETED")
print(f"{'='*80}\n")

return dp_result, dp_info

```

7.2 Initial State Extraction Flow

Function: `extract_descent_initial_state()`

```

def extract_descent_initial_state(cruise_results: CruiseResults,
                                  climb_fuel_kg: float,
                                  climb_time_s: float) -> DescentInitialState:
    """
    Extract initial descent state from cruise results.

    Args:
        cruise_results: Results from cruise simulation
        climb_fuel_kg: Fuel consumed during climb
        climb_time_s: Time spent in climb

    Returns:
        DescentInitialState object with extracted parameters
    """
    # Get final state from cruise
    final_altitude = float(cruise_results.altitude_m[-1])
    final_mach = float(cruise_results.mach_number[-1])
    final_weight = float(cruise_results.weight_kg[-1])

    # Total fuel consumed (climb + cruise)
    total_fuel_consumed = climb_fuel_kg + cruise_results.total_fuel_consumed_kg

    # Total time (climb + cruise)
    total_time = climb_time_s + cruise_results.total_time_s

    print(f"[DESCENT] Extracted initial state:")
    print(f"  Altitude: {final_altitude:.0f} m")
    print(f"  Mach: {final_mach:.3f}")
    print(f"  Weight: {final_weight:.1f} kg")
    print(f"  Total fuel consumed (climb+cruise): {total_fuel_consumed:.1f} kg")
    print(f"  Total time (climb+cruise): {total_time:.0f} s ({total_time/60:.1f} min)")

    return DescentInitialState(
        altitude_m=final_altitude,
        mach=final_mach,
        weight_kg=final_weight,
        fuel_consumed_total_kg=total_fuel_consumed,
        total_time_s=total_time
    )

```

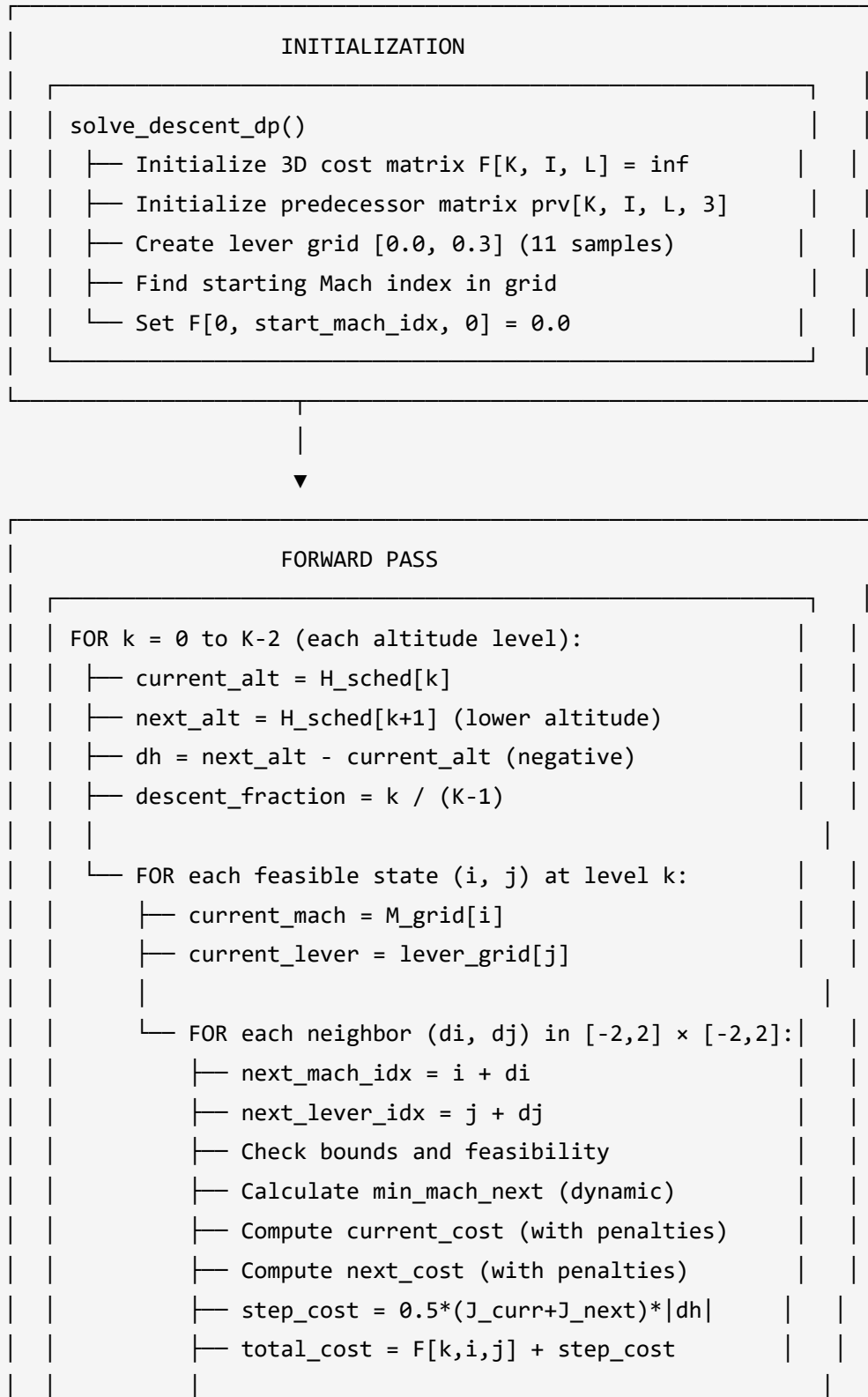
7.3 Dynamic Programming Optimization Flow

Function: `DescentCore.DynamicProgrammingOptimizer.solve_descent_dp()`

Detailed Flow:

DP OPTIMIZATION FLOW (DESCENT)

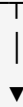
=====



```

└─ IF total_cost < F[k+1, next_i, next_j]:
    └─ F[k+1, next_i, next_j] = total_cost
    └─ prv[k+1, next_i, next_j] = [k,i,j]

```

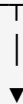


TERMINAL CONSTRAINT APPLICATION

```

Apply target Mach constraint:
└─ valid_final = |M_grid - target_mach| < tolerance
└─ FOR each Mach index i:
    └─ IF not valid_final[i]:
        └─ F[-1, i, :] = inf (infeasible)
    └─ Check if any path reached final altitude

```



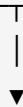
BACKTRACKING

```

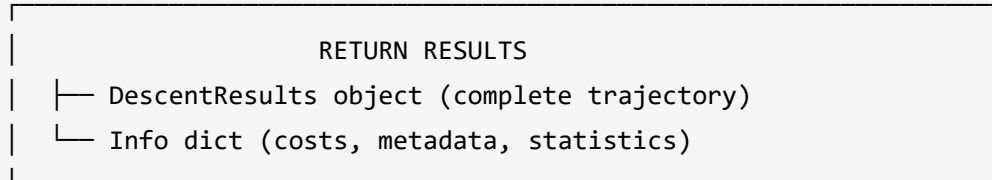
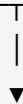
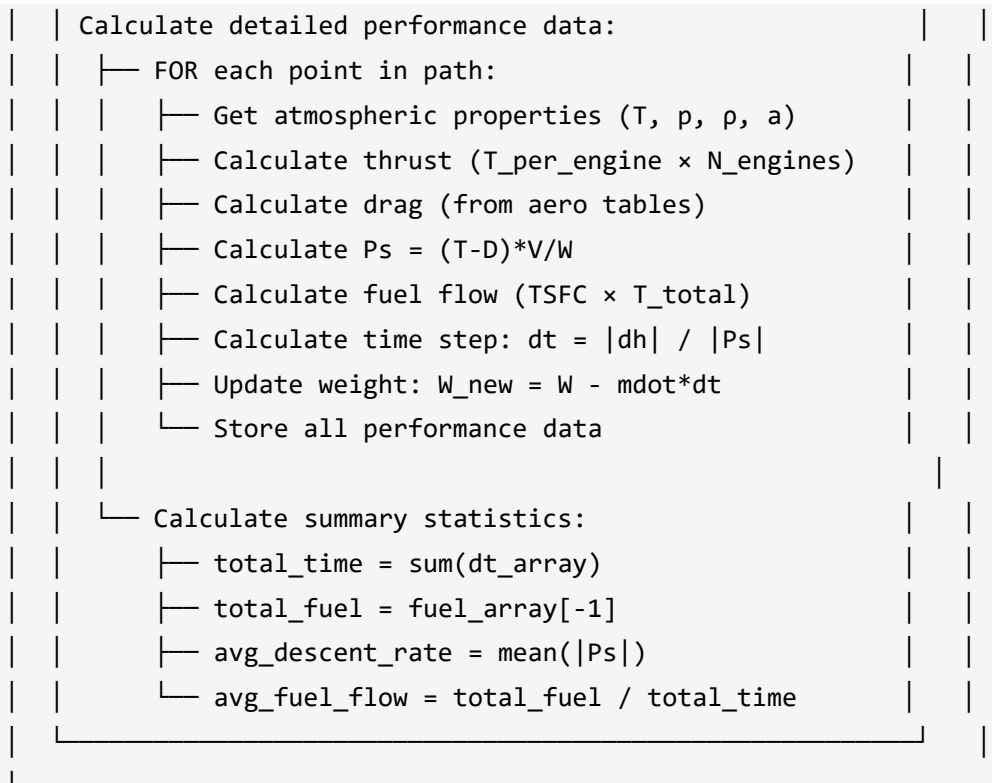
Find optimal final state:
└─ final_flat_idx = argmin(F[-1])
└─ (final_mach_idx, final_lever_idx) = unravel

└─ Backtrack using predecessor matrix:
    └─ current_state = [K-1, final_mach, final_lev]
    └─ WHILE current_state[0] >= 0:
        └─ Store state in path
        └─ current_state = prv[current_state]
    └─ Reverse path (start → finish)

```



TRAJECTORY RECONSTRUCTION



7.4 Function Call Hierarchy

```

main()
├─ run_descent_dp_optimization()
│   ├─ extract_descent_initial_state()
│   │   ├─ Access cruise_results final state
│   │   ├─ Calculate cumulative fuel (climb + cruise)
│   │   ├─ Calculate cumulative time (climb + cruise)
│   │   └─ Create DescentInitialState object
│   │
│   └─ Setup grids
│       ├─ calculate_min_descent_mach() → M_min
│       ├─ np.linspace(altitude) → H_descent
│       ├─ np.linspace(Mach) → M_grid
│       └─ np.linspace(lever) → lever_grid (in DP solver)
│
│   └─ DescentCore.DynamicProgrammingOptimizer.solve_descent_dp()
│       ├─ Initialize F, prv matrices
│       ├─ Set starting point F[0, start_mach_idx, 0] = 0.0
│       │
│       └─ Forward pass (FOR k in range(K-1)):
│           └─ FOR each feasible state (i, j):
│               └─ FOR each neighbor (di, dj):
│                   ├─ calculate_min_descent_mach() → min_mach_next
│                   ├─ DescentCore.compute_descent_cost() → current_cost
│                   │   ├─ a_from_altitude()
│                   │   ├─ eng.thrust_with_lever()
│                   │   ├─ aero.get_drag()
│                   │   └─ Calculate  $P_s = (T-D)*V/W$ 
│                   └─ eng.tsfc_current()
│                       └─ J_base =  $\dot{m} / |P_s|$ 
│                           ├─ PenaltySystem.compute_mach_penalty()
│                           ├─ PenaltySystem.compute_lever_penalty()
│                           └─ Return J_total = J_base + penalties
│                               └─ DescentCore.compute_descent_cost() → next_cost
│                                   └─ step_cost =  $0.5*(J_{curr} + J_{next}) * |dh|$ 
│                                       └─ Update F, prv if better path found
│                                           └─ [repeat for all neighbors]
│                                               └─ [repeat for all altitudes]
│
│       └─ Apply terminal Mach constraint

```

```

|         |   └─ Set F[-1, i, :] = inf for invalid Mach values
|         |
|         |─ Find optimal final state
|         |   └─ argmin(F[-1])
|         |
|         |─ Backtrack optimal path
|         |   └─ WHILE altitude_idx >= 0:
|         |       |   └─ Store current state
|         |       |       └─ current_state = prv[current_state]
|         |       └─ Reverse path
|         |
|         |─ Reconstruct trajectory
|         |   └─ FOR each point:
|         |       |   └─ isa_properties()
|         |       |   └─ a_from_altitude()
|         |       |   └─ eng.thrust_with_lever()
|         |       |   └─ aero.get_drag()
|         |       |   └─ Calculate Ps, fuel flow, dt
|         |       |   └─ Update weight
|         |       └─ Calculate summary statistics
|         |
|         └─ Return DescentResults, info dict
|
└─ Visualization
    └─ plot_descent_trajectory()
    └─ plot_descent_J_3d()
    └─ plot_complete_mission()

```

8) Integration and Interface

8.1 Mission Analysis Integration

Function: `extract_descent_initial_state()`

Integration Process:

```

def extract_descent_initial_state(cruise_results: CruiseResults,
                                climb_fuel_kg: float,
                                climb_time_s: float) -> DescentInitialState:
    """
    Extract initial descent state from cruise results.

    Integration point between cruise and descent phases. Extracts final
    cruise state and combines with cumulative mission data.

    Args:
        cruise_results: Complete cruise simulation results
        climb_fuel_kg: Fuel consumed during climb phase
        climb_time_s: Time spent in climb phase

    Returns:
        DescentInitialState: Initial conditions for descent optimization
    """
    # Get final state from cruise
    final_altitude = float(cruise_results.altitude_m[-1])
    final_mach = float(cruise_results.mach_number[-1])
    final_weight = float(cruise_results.weight_kg[-1])

    # Cumulative fuel: climb + cruise
    total_fuel_consumed = climb_fuel_kg + cruise_results.total_fuel_consumed_kg

    # Cumulative time: climb + cruise
    total_time = climb_time_s + cruise_results.total_time_s

    return DescentInitialState(
        altitude_m=final_altitude,
        mach=final_mach,
        weight_kg=final_weight,
        fuel_consumed_total_kg=total_fuel_consumed,
        total_time_s=total_time
    )

```

8.2 Main Interface Functions

Primary Optimization Interface:

```
def run_descent_dp_optimization(cruise_results: CruiseResults,
                               climb_fuel_kg: float,
                               climb_time_s: float,
                               aero: AeroTables,
                               engine: EngineWrapper,
                               target_altitude_m: float = TARGET_DESCENT_ALT_M,
                               target_mach: float = TARGET_APPROACH_MACH,
                               n_altitude_steps: int = 50,
                               n_mach_samples: int = 41,
                               lever_samples: int = 11) -> Tuple[DescentResults, Dict]:
    """
    Main interface for descent optimization.

    Runs 3D Dynamic Programming optimization with penalty guidance to find
    minimum fuel descent trajectory from cruise to approach.

    Returns:
        DescentResults: Complete optimal descent trajectory
        Dict: Additional optimization information and metadata
    """
```

Envelope Computation Interface:

```
def compute_full_descent_envelope(aero: AeroTables, eng: EngineWrapper,
                                  M_grid: np.ndarray, H_sched: np.ndarray,
                                  initial_weight_kg: float,
                                  lever_samples: int = 50,
                                  target_mach: float = TARGET_APPROACH_MACH) -> Tuple[np.ndarray,
                                                                                       np.ndarray,
                                                                                       np.ndarray]:
    """
    Compute full 3D descent cost envelope (J values) for visualization.

    Similar to climb's compute_full_engine_envelope, but for descent phase.
    Computes  $J = \text{mdot} / |P_s| + \text{penalties}$  across all (Mach, Altitude, Lever) combinations.

    Returns:
        J_grid_3d: 3D array of shape (len(M_grid), len(H_sched), lever_samples)
        lever_grid: 1D array of lever positions
    """
```

8.3 Backward Compatibility

Backward Compatibility Functions:

```
# Expose cost computation at module level
compute_descent_cost = DescentCore.compute_descent_cost

# Expose penalty system at module level
PenaltySystem = DescentCore.PenaltySystem

# Export configuration constants
MACH_TRAJECTORY_GUIDANCE = DescentCore.PenaltySystem.MACH_TRAJECTORY_GUIDANCE
LEVER_PENALTY_GUIDANCE = DescentCore.PenaltySystem.LEVER_PENALTY_GUIDANCE
TARGET_MACH_TOLERANCE = DescentCore.PenaltySystem.TARGET_MACH_TOLERANCE
```

8.4 Performance Metrics

Key Performance Indicators:

- **Fuel Consumption:** Total fuel burned during descent (typically 50-200 kg)
- **Descent Time:** Time required to reach approach altitude (typically 20-35 minutes)
- **Average Descent Rate:** Mean vertical velocity (typically 3-8 m/s or 600-1600 ft/min)
- **Fuel Efficiency:** Fuel consumed per meter of altitude lost (typically 0.005-0.020 kg/m)
- **Mach Trajectory:** Deceleration profile from cruise Mach to approach Mach
- **Lever Usage:** Throttle profile (typically idle 0-20% for optimal fuel efficiency)



9) Key Differences from Climb

9.1 Fundamental Differences

Flight Physics:

Aspect	Climb	Descent
Energy state	Energy addition	Energy dissipation

Aspect	Climb	Descent
Specific excess power	$P_s > 0$ (positive)	$P_s < 0$ (negative)
Thrust-drag balance	$T > D$ (thrust exceeds drag)	$T < D$ (drag exceeds thrust)
Altitude direction	Low $\rightarrow$ High (300m $\rightarrow$ 10,000m)	High $\rightarrow$ Low (10,000m $\rightarrow$ 300m)
Cost denominator	P_s	,
Feasibility check	$P_s > 0$ required	$P_s < 0$ required

Optimization Objectives:

Aspect	Climb	Descent
Starting state	Low altitude, low Mach	High altitude, high Mach
Target state	High altitude, cruise Mach	Low altitude, approach Mach
Mach trajectory	Acceleration (0.25 $\rightarrow$ 0.78)	Deceleration (0.78 $\rightarrow$ 0.25)
Optimal thrust	Moderate-high (50-80%)	Idle-low (0-20%)
Primary constraint	Thrust availability	Drag sufficiency
Lever grid range	0.0 $\rightarrow$ 1.0 (full range)	0.0 $\rightarrow$ 0.3 (idle range)

9.2 Penalty System Differences

Mach Penalty:

Aspect	Climb	Descent
Target location	High Mach at top	Low Mach at bottom
Corridor orientation	Opens upward (accelerating)	Narrows downward (decelerating)
Physical challenge	Gain speed while climbing	Lose speed while descending

Aspect	Climb	Descent
Penalty logic	Encourage speed gain	Encourage speed loss
Corridor bounds	Around high target Mach	Around low target Mach

Lever Penalty:

- **IDENTICAL** in both climb and descent
- Engine limits are altitude-independent
- Penalizes high thrust (>85%) equally in all phases
- Effect differs: Climb naturally needs moderate thrust, descent naturally uses idle

9.3 Grid and Discretization Differences

State Space:

Parameter	Climb	Descent
Altitude range	[300m, 10,000m]	[10,000m, 300m]
Altitude steps	50 (ascending)	50 (descending)
Mach range	[0.25, 0.78] typical	[0.25, 0.85] typical
Mach samples	81	41
Lever range	[0.0, 1.0] full	[0.0, 0.3] idle-focused
Lever samples	50	11

Computational Complexity:

- **Climb:** $50 \times 81 \times 50 = 202,500$ states
- **Descent:** $50 \times 41 \times 11 = 22,550$ states (9× fewer)

9.4 Performance Metrics Comparison

Typical Values (A320-class aircraft, 10,000m climb/descent):

Metric	Climb	Descent
Fuel consumption	1,500-2,000 kg	50-200 kg
Time duration	15-25 minutes	20-35 minutes
Average rate	8-15 m/s (1,600-3,000 ft/min)	3-8 m/s (600-1,600 ft/min)
Distance covered	50-80 km	150-250 km
Fuel per meter	0.15-0.20 kg/m	0.005-0.020 kg/m
Mission fuel fraction	30-40%	1-3%

9.5 Strategy System Differences

Climb:

- Has comprehensive strategy system (StrategyManager)
- Multiple strategies: Linear, Exponential, Constant Speed, Constant Mach
- Energy allocation between climb rate and speed
- Strategy simulation alongside DP optimization

Descent:

- No strategy system (DP optimization only)
- Single approach: Minimum fuel via 3D DP
- Idle descent inherently optimal for fuel
- Focus on penalty-guided trajectory shaping

9.6 Module Structure Comparison

Similarities:

- Both use `DescentCore` / `ClimbingCore` architecture
- Both have `PenaltySystem` with Mach and Lever penalties
- Both have `DynamicProgrammingOptimizer`
- Both integrate with `AeroTables` and `EngineWrapper`
- Both use ISA atmosphere model

Differences:

- Climb has `StrategyManager` (descent does not)
 - Climb has `EnergyCalculator` for strategies (descent does not)
 - Descent has `calculate_min_descent_mach()` (climb uses fixed limits)
 - Descent focuses on deceleration (climb on acceleration)
 - Descent uses narrower lever range (idle-focused)
-

Summary

The descent module provides a fuel-optimal trajectory optimization system that:

✓ Core Capabilities

- **3D Dynamic Programming** with altitude, Mach, and lever state variables
- **Penalty-guided optimization** for realistic Mach deceleration profiles
- **Dynamic minimum Mach** calculation based on stall speed and weight
- **Terminal constraints** to achieve target approach speed (Mach 0.25 at 300m)
- **Mission integration** with seamless connection to climb and cruise phases

✓ Technical Features

- **Negative specific excess power** handling for energy dissipation
- **Reachability corridor** ensuring target Mach is achievable
- **Idle-focused lever grid** (0-30%) for fuel-efficient descent
- **Trapezoidal integration** for accurate fuel cost calculation
- **Backtracking algorithm** for optimal path extraction

✓ Key Results

- **Minimal fuel consumption:** Typically 50-200 kg for 10,000m descent
- **Idle thrust preferred:** Optimizer naturally selects 0-20% lever
- **Longer duration:** 20-35 minutes (slower than climb due to idle thrust)
- **Greater distance:** 150-250 km ground distance during descent

- **Mission contribution:** ~1-3% of total mission fuel

✓ **Critical Differences from Climb**

- **Energy dissipation** vs. energy addition
- **Deceleration** vs. acceleration
- **Idle thrust optimal** vs. moderate-high thrust required
- **$P_s < 0$ feasibility** vs. $P_s > 0$ feasibility
- **No strategy system** (DP only) vs. multiple strategies

✓ **Academic Standards**

- Professional documentation following academic conventions
 - Validated physics with energy conservation principles
 - Comprehensive mathematical formulation
 - Industry-relevant assumptions and constraints
 - Research-grade optimization methodology
-

Last Updated: October 2025

Version: 3.0 (Complete Rewrite Matching Climb Documentation Structure)

Author: Mission Analysis System